

Contents

SQL Validation Scenarios	1
Purpose	1
How To Use	2
Scenario Coverage	2
Review Notes	2
Aggregation Scenario	2
Business Request	2
Tiny Generic Schema Context	2
Expected Chain To Use	3
Expected Copilot Behaviors	3
Unacceptable Copilot Behaviors	3
Manual Pass/Fail Checklist	3
CTE Validation Scenario	4
Business Request	4
Tiny Generic Schema Context	4
Expected Chain To Use	4
Expected Copilot Behaviors	4
Unacceptable Copilot Behaviors	4
Manual Pass/Fail Checklist	5
Daily Morning Extract Scenario	5
Business Request	5
Tiny Generic Schema Context	5
Expected Chain To Use	5
Expected Copilot Behaviors	6
Unacceptable Copilot Behaviors	6
Manual Pass/Fail Checklist	6
Join Selection Scenario	6
Business Request	6
Tiny Generic Schema Context	7
Expected Chain To Use	7
Expected Copilot Behaviors	7
Unacceptable Copilot Behaviors	7
Manual Pass/Fail Checklist	7

SQL Validation Scenarios

Purpose

These scenarios are manual validation checks for SQL prompt chains. A human reviewer can paste each scenario into Copilot and confirm whether Copilot uses the intended chain behavior before writing or accepting SQL.

The scenarios are not automated database tests. They do not require real database access, creden-

tials, private schemas, or production data. They are designed to check whether Copilot asks for the right context, maps logic before SQL, identifies correctness risks, and provides manual checks the analyst can run locally.

How To Use

1. Open one scenario file.
2. Paste the business request and schema context into Copilot.
3. Tell Copilot to follow the expected prompt chain named in the scenario.
4. Compare Copilot's response to the expected and unacceptable behaviors.
5. Complete the manual pass/fail checklist.

The reviewer should fail a scenario if Copilot skips the required mapping, chooses SQL too early, ignores grain or duplicate risk, invents schema details, or claims it has validated results without analyst-provided evidence.

Scenario Coverage

Scenario	Prompt chain exercised
cte-validation-scenario.md	prompt-chains/sql-cte-validation/
join-selection-scenario.md	prompt-chains/sql-joins/
aggregation-scenario.md	prompt-chains/sql-aggregation/
daily-morning-extract-scenario.md	prompt-chains/sql-general/, routed into prompt-chains/sql-logi

Review Notes

- Keep the scenarios generic. Do not add private company schema, credentials, server names, account numbers, customer records, or real data.
- Do not paste long content from prompt chains or reference drawers into these scenarios.
- Treat each checklist as a human judgment aid, not a substitute for running local SQL checks in the analyst's own environment.

Aggregation Scenario

Business Request

An analyst needs a monthly summary by store showing total completed sales, number of distinct buying customers, average completed sale amount, and the count of refunded transactions. Only stores with at least 10 completed sales for the month should appear.

Ask Copilot to design the aggregation before writing final SQL.

Tiny Generic Schema Context

Table	Important columns
stores	store_id, store_name, region_code, status
transactions	transaction_id, store_id, customer_id, transaction_date, transaction_status, sale_am

Table	Important columns
regions	region_code, region_name

Expected output grain: one row per store per calendar month.

Known risk: joining to any one-to-many detail table before aggregation could inflate sales totals or customer counts.

Expected Chain To Use

Use `prompt-chains/sql-aggregation/`.

Expected Copilot Behaviors

- Defines each metric before SQL, including numerator, eligible rows, and null handling.
- Confirms output grain as store-month.
- Distinguishes **WHERE** filters from **HAVING** rules, especially the completed sales threshold.
- Uses conditional aggregation for refunded transaction counts without mixing refunded amounts into completed sales.
- Uses `COUNT(DISTINCT customer_id)` for distinct buying customers if a customer can have multiple transactions.
- Proposes reconciliation checks for totals, distinct counts, group counts, date boundaries, and duplicate amplification.

Unacceptable Copilot Behaviors

- Drafts grouped SQL before defining the metric map and output grain.
- Counts all transaction statuses as completed sales.
- Uses `COUNT(customer_id)` where distinct customers are requested.
- Places the 10 completed sales threshold in a row-level **WHERE** filter instead of a group-level rule.
- Treats null `sale_amount` values as zero without confirming that business rule.
- Claims aggregation correctness without reconciliation checks.

Manual Pass/Fail Checklist

Check	Pass/Fail
Copilot used or clearly followed the aggregation chain.	
Copilot defined output grain before SQL.	
Copilot mapped all requested metrics before SQL.	
Copilot separated completed sales from refunded transactions.	
Copilot used or recommended distinct customer counting.	
Copilot placed the completed-sales threshold at group level.	
Copilot proposed manual reconciliation and duplicate-risk checks.	
Copilot avoided private schema details, credentials, and real data.	

CTE Validation Scenario

Business Request

An analyst has a multi-CTE Oracle SQL query that produces one row per active customer for the current month. The query first filters customers, then summarizes monthly orders, then joins the summary to customer attributes.

Ask Copilot to validate whether the CTE stages are logically correct before any rewrite is suggested.

Tiny Generic Schema Context

Table	Important columns	Notes
customers	customer_id, status, signup_date, region_code	One row per customer. stat
orders	order_id, customer_id, order_date, order_status, order_amount	Many rows per customer. C
regions	region_code, region_name	One row per region code.

Expected output grain: one row per active customer with current-month order count, current-month order amount, and region name.

Known risk: joining **orders** before aggregation may duplicate customer rows.

Expected Chain To Use

Use `prompt-chains/sql-cte-validation/`.

Expected Copilot Behaviors

- Builds an inventory of every CTE before proposing any rewrite.
- States each CTE's purpose, source tables, dependencies, output fields, and expected grain.
- Identifies where filters belong, especially active-customer status, current month date boundaries, and cancelled-order exclusion.
- Proposes stage-level validation checks such as row counts, duplicate checks by **customer_id**, current-month boundary checks, and null checks for region.
- Separates correctness validation from performance or style recommendations.
- Asks for the actual SQL if it has not been provided, instead of inventing a full query.

Unacceptable Copilot Behaviors

- Immediately rewrites the SQL before mapping CTE stages.
- Assumes private table names, hidden columns, database credentials, or actual production row counts.
- Treats the final output as validated without analyst-run evidence.
- Ignores output grain or duplicate risk from the orders relationship.
- Collapses the CTEs into one query without explaining whether stage-level validation is preserved.

Manual Pass/Fail Checklist

Check	Pass/Fail
Copilot used or clearly followed the CTE validation chain.	
Copilot created a CTE-stage map before rewrite advice.	
Copilot stated expected grain for each stage and the final output.	
Copilot identified duplicate risk from many orders per customer.	
Copilot proposed manual validation checks the analyst could run locally.	
Copilot did not claim database validation without evidence.	
Copilot did not invent non-generic schema details or real data.	

Daily Morning Extract Scenario

Business Request

An analyst needs a daily morning extract for operations. Each weekday morning, the extract should list open service tickets created yesterday, include the assigned team and customer priority, exclude test tickets, and flag tickets that have no assigned owner. The query should be readable and efficient enough for a recurring scheduled job.

Ask Copilot to classify the request with the general SQL chain, route to business-logic mapping where needed, draft efficient SQL only after the mapping is clear, and propose result-validation checks before the extract is accepted.

Tiny Generic Schema Context

Table	Important columns	Notes
service_tickets	ticket_id, customer_id, assigned_user_id, created_at, status, is_test_ticket	One
customers	customer_id, customer_name, priority_code	One
users	user_id, team_id, user_name, active_flag	One
teams	team_id, team_name	One

Expected output grain: one row per qualifying service ticket.

Known risks: date-window boundaries for "yesterday", optional assigned owner, test-ticket exclusion, and accidental row loss from joins.

Expected Chain To Use

Start with `prompt-chains/sql-general/`.

Expected routing:

- Use `prompt-chains/sql-logic-mapping/` to clarify business rules, field mappings, output grain, date window, exclusions, and owner flag logic.
- Use `prompt-chains/sql-efficiency/` to draft readable Oracle SQL after the logic map is accepted and to review avoidable inefficiency.

- Use `prompt-chains/sql-result-validation/` once SQL exists or is being finalized, especially for row counts, date boundaries, duplicates, null owner checks, and join row preservation.

Expected Copilot Behaviors

- Classifies the request as a broad SQL build that needs routing rather than a single immediate final query.
- Builds a logic map before SQL, including status rules, yesterday date window, test-ticket exclusion, owner flag, joins, and output columns.
- Preserves tickets with missing assigned users by using optional joins where appropriate.
- Drafts Oracle SQL only after the mapping is clear, using date predicates that can use an index on `created_at`.
- Reviews efficiency risks such as functions applied to indexed date columns in row filters, unnecessary selected columns, and duplicate-prone joins.
- Proposes result-validation checks for row counts, one row per `ticket_id`, yesterday boundaries, excluded test tickets, open status inclusion, missing owner counts, and unmatched customer or team joins.

Unacceptable Copilot Behaviors

- Skips classification and routing and immediately returns final SQL.
- Invents hidden scheduling tables, credentials, private operational fields, or real ticket data.
- Uses joins that drop unassigned tickets even though missing owners must be flagged.
- Uses vague date logic that does not define yesterday's start and end boundaries.
- Treats runtime efficiency as plan-based tuning without any plan evidence.
- Claims result correctness without analyst-run validation evidence.

Manual Pass/Fail Checklist

Check	Pass/Fail
Copilot started from or clearly followed the general SQL chain.	
Copilot routed to logic mapping before final SQL.	
Copilot routed to efficiency review for the recurring extract query.	
Copilot routed to result validation once SQL existed or was being finalized.	
Copilot preserved unassigned tickets and produced an owner-missing flag.	
Copilot defined yesterday's date boundaries clearly.	
Copilot proposed row-count, duplicate, boundary, exclusion, and null checks.	
Copilot avoided private schema details, credentials, and real data.	

Join Selection Scenario

Business Request

An analyst needs a query that lists every active product and shows its latest approved price if one exists. Products without an approved price should still appear so the analyst can find missing pricing.

Ask Copilot to choose and explain the correct joins before drafting final SQL.

Tiny Generic Schema Context

Table	Important columns	Notes
products	product_id, product_name, status, category_id	One row per product
price_history	price_id, product_id, effective_date, price_amount, approval_status	Many rows per product
categories	category_id, category_name	One row per category

Expected output grain: one row per active product.

Known risk: multiple approved prices per product can create duplicates unless the latest approved row is selected before joining.

Expected Chain To Use

Use `prompt-chains/sql-joins/`.

Expected Copilot Behaviors

- Confirms that active products are row-preserved.
- Chooses a left join from active products to the latest approved price so products without approved prices remain in the result.
- Explains why an inner join would drop products without prices.
- Addresses the one-to-many relationship from products to price history.
- Requires a latest-price selection method before the final join, such as a ranked subquery or pre-filtered CTE.
- Proposes duplicate checks by `product_id` and unmatched-row checks for products with no approved price.

Unacceptable Copilot Behaviors

- Uses an inner join without noting that missing-price products will be dropped.
- Joins directly to all approved price rows and ignores duplicate risk.
- Applies a right or full join without business justification.
- Moves approved-price filters into a `WHERE` clause in a way that cancels row preservation after a left join.
- Claims the join is correct without row-count, duplicate, or unmatched-row validation checks.

Manual Pass/Fail Checklist

Check	Pass/Fail
Copilot used or clearly followed the joins chain.	
Copilot stated which table must be row-preserved.	
Copilot selected a left join for optional price data.	
Copilot explained why an inner join would be wrong for this request.	
Copilot addressed latest-approved-price selection before joining.	

Check	Pass/Fail
Copilot proposed duplicate and unmatched-row checks.	
Copilot avoided private schema details, credentials, and real data.	